Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Робототехника и комплексная автоматизация»

КАФЕДРА «Компьютерные системы автоматизации производства»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

«Система ориентации робокара на местности с помощью видеокамеры»

Студент РК9-84Б _____ Ольхов Е.О.
(Подпись, дата)

Руководитель ВКР _____ Буханов С.А.
(Подпись, дата)

Нормоконтролер _____ Гавриленков С.И.
(Подпись, дата)

2022 г.

РЕФЕРАТ

Выпускная квалификационная работа выполнена по теме: «Система ориентации робокара на местности с помощью видеокамеры.».

Расчетно-пояснительная записка содержит 60 листов формата А4, 18 рисунков и 2 таблицы.

Целью выполнения работы является создание модуля локализации беспилотной складской тележки (Робокара). В работе проведен анализ существующих алгоритмов локализации мобильного робота, выявление недостатков и доработка под существующие задачи. В ходе работы описывается разработка самой системы: разработка алгоритма локализации роботом на заданной карте, разработка модулей, отвечающих за взаимодействие между роботом и датчиками, тестирование модуля локализации. Заключение посвящено основным выводам по проделанной работе.

Ключевые слова: Система локализации, Ros, Python, Фильтр частиц, Мобильный робот.

Объектом автоматизации является гибкая производственная линия.

СОДЕРЖАНИЕ

РЕФЕРАТ 2
СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ 5
ВВЕДЕНИЕ 7
1. Анализ объекта автоматизации 9
1.1. Анализ существующей гибкой производственной системы 9
2. Концептуальное проектирование системы 12
2.1. Постановка задачи 12
2.2. Выбор оптимального метода определения положения 12
2.3. Анализ существующего технического решения робокар 14
2.4. Выбор оптимального алгоритма 15
2.4.1 Описание метода ekf-slam 17
2.4.2 Описание метода основанного на графе (graph-based slam) 19
2.4.3 Описание метода основанного на фильтре частиц 20
2.4.4 Выбор алгоритма реализации системы ориентации робокара 21
2.5 Выбор платформы для работы с робокаром 21
2.6 Выбор языка программирования 22
3. Техническое задание 24
3.1 Общие сведения 24
3.2 Назначение и цели создания системы 24
3.3 Требования к системе в целом 25
4. Структурное проектирование 26
4.1. Выбор технической базы 26
4.2. Представление карты местности 27
4.3. Алгоритм 29
5. Рабочее проектирование 32
5.1. Построение и настройка карты 32
5.2. Разработка модуля ориентации робокара 34
6. Апробирование 39

6.1. Тестирование работы алгоритма 39
ЗАКЛЮЧЕНИЕ 46
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 47
Приложение А 49
Приложение Б 54
Приложение В 57
Приложение Г 58

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

ГПС – совокупность в разных сочетаниях оборудования с ЧПУ, роботизированных технологических комплексов, гибких производственных модулей, отдельных единиц технологического оборудования и систем обеспечения их функционирования в автоматическом режиме в течение заданного интервале времени, обладающую свойством автоматизированной переналадки при производстве изделий произвольной номенклатуры в установленных пределах значений их характеристик.

Python – высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ.

Ros – это экосистема для программирования роботов, предоставляющая функциональность для распределённой работы. ROS обеспечивает стандартные службы операционной системы, такие как: аппаратную абстракцию, низкоуровневый контроль устройств, реализацию часто используемых функций, передачу сообщений между процессами, и управление пакетами. ROS основан на архитектуре графов, где обработка данных происходит в узлах, которые могут получать и передавать сообщения между собой. Библиотека ориентирована на Unix-подобные системы (Ubuntu Linux включен в список «поддерживаемых», в то время как другие варианты, такие как Fedora и Mac OS X, считаются «экспериментальными»)

SLAM – Метод, используемый в мобильных автономных средствах для построения карты в неизвестном пространстве или для обновления карты в заранее известном пространстве с одновременным контролем текущего местоположения и пройденного пути. Популярные методы приближённого решения данной задачи - фильтр частиц и расширенный фильтр Калмана. Некоторые реализации метода используются в беспилотных автомобилях,

летательных аппаратах, автономных подводных аппаратах, планетоходах, и даже внутри человеческого тела.

ВВЕДЕНИЕ

В настоящее время большое количество производственных предприятий стремится максимально автоматизировать процесс на каждом возможном этапе работы. Широкое применение в промышленности получили мобильные роботы, которые решают целый спектр задачи по автоматизации работы предприятий

Роботизация склада выгодна по многим факторам Основные преимущества автоматизированных погрузчиков:

1) Повышение производительности. Робот справляется с любой складской операцией в среднем в 3,5–4 раза быстрее человека.
2) Робот поднимает и переносит тяжелый груз, с которым человек не может справиться без подъемных механизмов.
3) Миниатюрные складские погрузчики не требуют больших проходов, что позволяет экономить полезную площадь.
4) Исключение человеческого фактора. Это приводит к устранению ошибок в сортировке грузов и травмирования людей. Существенно снижается риск случайной порчи товара. Исключение травм у работников сокращает расходы на компенсации причиненного ущерба здоровью.
5) Робот способен работать без отдыха и в длительном режиме.

Автоматизация складов доказала свою эффективность и экономическую выгодность на крупных складах с активным движением товаров. Складские роботы позволяют высвободить большое количество рабочих, сократить потребность во временных работниках в напряженные периоды, значительно увеличить товарооборот за счет повышения скорости обработки заказов.

Роботизация складов становится неизбежной перспективой во всем мире, в т. ч. в РФ. Это связано с укрупнением компаний, переходом их на складские

помещения с большими площадями, активным развитием онлайн-торговли. По прогнозам, к 2025 году более 20 % компаний в Европе и до 90 % в Америке внедрят роботов для упаковки, комплектации, транспортировки грузов, управления запасами и оптимизации логистических схем. В России прогнозы гораздо скромнее, но и здесь роботизация должна достигнуть 7–8 % в Московском регионе и 3–4 % в среднем по стране

Широкое применение находят для разнообразных беспилотников. Использование автономных транспортных аппаратов имеет широкий спектр применения в самых разных сферах деятельности человека. С их помощью можно полностью или частично исключить человеческий фактор при перемещении грузов и гарантировать высокую сохранность и скорость транспортировки. Важную роль в работе каждого беспилотного аппарата является точное распознавание окружающих объектов и локализация робота. Для получения пространственных данных, с целью составления и оформления карт используют различные алгоритмы основанные на машинном обучении

1. Анализ объекта автоматизации

1.1. Анализ существующей гибкой производственной системы

Исследуемым объектом является гпс производства фирмы denford, в частности один из модулей ее транспортно-накопительной системы – робокар, расположенный в лабораторной аудитории 297 кафедры «компьютерные системы автоматизации производства» МГТУ им. Н. Э. Баумана. Структурная схема гпс denford приведена на рисунке 1. [1]

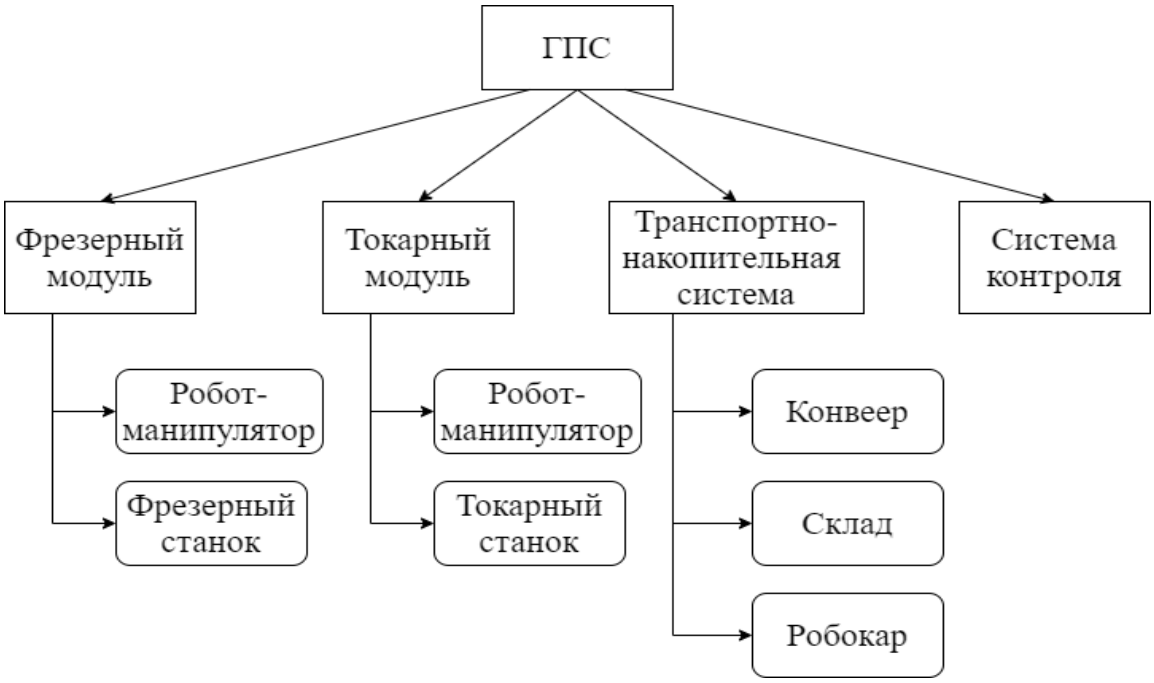

Рисунок 1 – структурная схема гпс denford

Робокар или автоматически управляемая тележка (agv – automatic guided vehicle) – транспортер с электроприводом, предназначенный для перемещения грузов. Тележка автоматическая, а это значит, что для ее обслуживания не нужен отдельный оператор - тележки двигаются по заданной траектории в автономном режиме без участия человека. Agv снабжена всеми необходимыми системами и элементами безопасности, может эксплуатироваться на вредных или опасных производствах, местах скопления людей и других двигающихся agv.

Основные направления работы:

1) Движение по предварительно определенной траектории различной формы, включая развилки;
2) Поддержание постоянной скорости в случае сопровождения конвейера или других объектов;
3) Остановки и продолжение движения, как в заранее определенных позициях, так и "по требованию" оператора;
4) Загрузка и разгрузка перевозимых компонентов;
5) Беспроводная связь с другими agv или центральным терминалом для создания сети тележек, движущихся без участия людей;
6) Картографирование неподвижных препятствий для проезда на минимальном расстоянии;
7) Распознавание перемещающихся препятствий, ожидание их исчезновения и продолжение работы.
8) Характеристики робокара:
    1) Вес тележки – 110 кг;
    2) Размеры – 1000x550x375;
    3) Минимальный радиус поворота – 1 м;
    4) Средняя скорость – 30 м/мин;
    5) Среднее время работы – 8 ч;
    6) Главный тяговый двигатель - двигатель постоянного тока с постоянным магнитом, приводимый в действие 2-квадрантной системой привода fet;
    7) Рулевой двигатель - шаговый двигатель, управляемый шаговым приводом denstep;
    8) Аккумулятор - герметичные свинцово-кислотные батареи;
    9) Вход - 250–265 В переменного тока 50/60 гц;
    10) Выход - 25,0–30,0 В 0,8 а;
    11) Вес аккумулятора – 0.8 кг;

Исследуемая тележка имеет инфракрасный датчик стыковки и датчик столкновений

Основные проблемы существующего решения: необходимость постоянного присутствия человека при принятии и транспортировке грузов, устаревшее техническое состояние робокара и отсутствие удаленного управления. Данная система не способна автономно перемещать товары и подстраиваться под состояние окружающего пространства

2. Концептуальное проектирование системы

2.1. Постановка задачи

Цель работы: разработка системы ориентации робокара на местности с помощью видеокамеры

В ходе разработки проекта были поставлены следующие **задачи**:

1) Сформулировать требования, предъявляемые к разрабатываемой системе;
2) Выбор оптимального метода определения положения;
3) Проанализировать технические характеристики робокара и реализовать систему при минимальных технических доработках;
4) Выбор оптимального алгоритма в поставленных условиях;
5) Разработать алгоритмы основных функций программ;
6) Разработать исходный код программной части на языке python на базе открытой робототехнической платформы ros;

2.2. Выбор оптимального метода определения положения

Современные автономные складские робокары имеют множество различных решений по транспортировке грузов. Одной из наиболее острых проблем, стоящих перед современными роботами, является навигация в больших помещениях. Существующие методы определения положения можно разделить на глобальные и локальные. Глобальные методы используют для определения текущего местоположения на обширном пространстве. Локальными способами определения текущего местоположения являются такие, которые позволяют очень точно определить текущее местоположение в относительно небольшом пространстве.

Глобальные методы:

- Gps не работает в помещении, когда не видны спутники. Нет возможности определения этажа в многоэтажном здании. – оптические системы определение текущего местоположения по ориентирам, выявленными при помощи машинного зрения является наиболее перспективным и сложным способом.
- bluetooth/wifi beacons или данные о сигналах сбор данных о трех и более ближайших bluetooth/wifi точках позволяет рассчитать примерное местонахождение относительно этих точек.

Локальные методы:

- Лидар – лазерный радар, постоянно считывающий расстояния до окружающих препятствий при помощи лазерного дальномера. Довольно хорошая точность. Не требуется установка дополнительных маяков.
- одометрия – использование данных о движении приводов для оценки перемещения. Очень хорошая точность на малых расстояниях, пока нет препятствий и пока имеется хорошее сцепление колес с полом. Без корректировки ошибка накапливается очень быстро и делает вычисление положения невозможным.

В условиях существующей системы небольшого замкнутого складского помещения локальные методы картографирования позволят точнее определить местоположение робокара. В качестве оптимального решения задачи выбран лидар. **Лидар** — технология получения и обработки информации об удалённых объектах с помощью активных оптических систем, использующих явления отражения света и его рассеивания в прозрачных и полупрозрачных средах.[2]

Лидар как прибор представляет собой, как минимум, активный дальномер оптического диапазона. Сканирующие лидары в системах машинного зрения формируют двумерную или трёхмерную картину окружающего пространства.

Принцип действия лидара не имеет больших отличий от радара: направленный луч источника излучения отражается от целей, возвращается к

источнику и улавливается высокочувствительным приёмником (в случае лидара — светочувствительным полупроводниковым прибором); время отклика прямо пропорционально расстоянию до цели.

В отличие от радиоволн, эффективно отражающихся только от достаточно крупных металлических целей, световые волны подвержены рассеиванию в любых средах, в том числе в воздухе, поэтому возможно не только определять расстояние до непрозрачных (отражающих свет) дискретных целей, но и фиксировать интенсивность рассеивания света в прозрачных средах. Возвращающийся отражённый сигнал проходит через ту же рассеивающую среду, что и луч от источника, подвергается вторичному рассеиванию, поэтому восстановление действительных параметров распределённой оптической среды — достаточно сложная задача, решаемая как аналитическими, так и эвристическими методами.

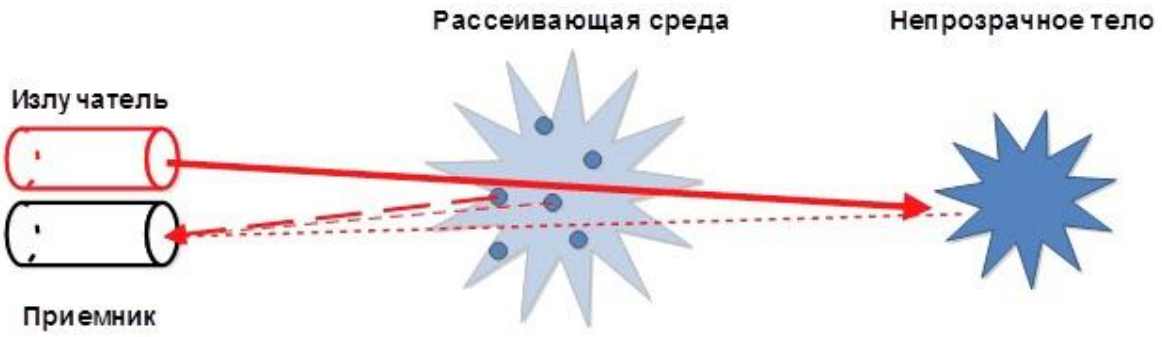

Рисунок 2 – Принцип действия лидара

2.3. Анализ существующего технического решения робокар

Исследуемый робокар производства фирмы denford использует для распознавания окружающих объектов инфракрасный датчик, технические характеристики которого не позволяют с достаточной точностью и полнотой исследовать и составлять карту. В качестве альтернативного решения можно

использовать лидар . Это решение более эффективно в плане скорости работы, точности и простоты реализации алгоритмов.

Алгоритм было принято реализовать на базе открытой робототехнической платформы ros. Ros обеспечивает стандартные службы операционной системы, такие как: аппаратную абстракцию, низкоуровневый контроль устройств, реализацию часто используемых функций, передачу сообщений между процессами, и управление пакетами. Ros основан на архитектуре графов, где обработка данных происходит в узлах, которые могут получать и передавать сообщения между собой. Библиотека ориентирована на unix-подобные системы

На текущей версии робокар установлена печатная плата arduino. При использовании arduino для робототехнических проектов существует одно существенное ограничение: arduino не может быть использован как полноценный вычислительный узел, который может обрабатывать все операции, требуемые для нормального функционирования робота. Наилучшим решением будет использование одноплатного компьютера raspberry pi. Существующие библиотеки на ros для работы с лидаром, , такие как /rplidar. Позволяют максимально эффективно и быстро реализовывать задуманные решения. В результате анализа существующего технического решения было выявлено, что для оптимальной реализации поставленной задачи необходимо усовершенствовать техническую часть робокара. Было предложено решение об использовании одноплатного компьютера raspberry pi в паре с лидаром.

2.4. Выбор оптимального алгоритма

Для решения задачи ориентации робокара хорошо подходит метод slam [3]– с англ. «simultaneous location and mapping» - одновременное построение карты и локализация на ней. Эта технология, помогает роботу обнаружить себя в пространстве, при этом постоянно обновляя данные о ней. В настоящее время

существует большое количество вариантов решения задачи slam. Решение задачи одновременного картографирования и локализации можно разделить на несколько подзадач. (рис. 4)

1) **Вычисление текущего положения** робокара на основе одометрических датчиков
2) **Поиск ключевых точек** по которым можно определить расположение препятствий в рамках заданной области. В качестве ключевых точек удобно использовать ориентиры (англ. Landmark), которые часто встречаются в пространстве и которые не сложно обнаружить (углы стен, прямые линии, контрастные или яркие точки). В ходе работы, повторно встретив ключевую точку алгоритм должен однозначно ее идентифицировать. В результате чего об одной точке могут быть получены данные из нескольких положений робота в пространстве.
3) **Решение задачи ассоциации (англ. Data association).** Если новую ключевую точку можно поставить в соответствие с уже существующей, то вес существующей точки увеличивается. В противном случае новая точка добавляется в общее облако точек.
4) **Хранение карты.** Собранное облако точек сохраняется и обновляется на основе актуальных данных с сенсоров.

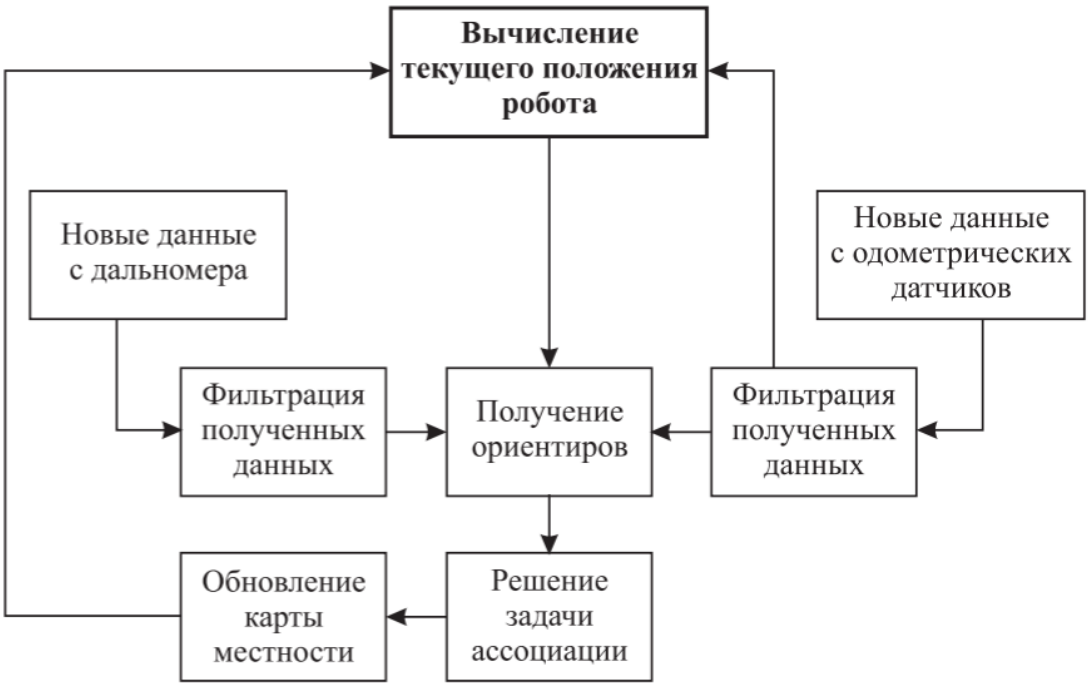

Рисунок 3 – Схема slam-алгоритма

Каждая из вышеописанных подзадач имеет разные варианты реализации. Исходя из теоретических подходов к решению slam, его разделяют на несколько категорий, в зависимости от методов решения основных подзадач.

Основными парадигмами при решении slam задач принято считать алгоритмы, основанные на расширенном фильтре калмана (extended kalman filter), фильтре частиц (fast slam), а так же алгоритмы основанные на оптимизации графов (graph-based slam). Рассмотрев подробнее каждый из приведенных методов определим оптимальный для решения поставленной задачи.

2.4.1 Описание метода ekf-slam

Ekf-slam[4] основан на применении расширенного фильтра калмана, применяемого при решении задачи локализации. Расширенный фильтр калмана является нелинейной версией фильтра калмана, рекурсивного фильтра, оценивающего вектор состояния динамической системы, используя серию

неполных и зашумленных измерений. Фильтр калмана выполняет вычисления с помощью значений предшествующих аргументов, переоценки вектора состояния заранее известной динамической системы, т.е. Для расчёта текущего состояния системы необходимо знать текущее измерение, а также предыдущее состояние самого фильтра. Однако расширенный фильтр калмана (ekf) отличается от простого фильтра калмана тем, что он может быть использован для оценки состояния нелинейных систем .

Расширенный фильтр калмана используется для оценки состояния (положения) робота от данных одометрии и ориентиров[4]. Стоит заметить, что у него нет обновления карты, которое необходимо при использовании ekf-slam. Кроме этого, метод ekf-slam использует расширенный вектор состояния yt, который включает положение робота xt и положение всех элементов карты mi:

Для расчёта текущего состояния системы необходимо знать текущее измерение, а также предыдущее состояние самого фильтра. Состояние фильтра находится в двух переменных:

yt – оценка вектора состояния динамической системы в момент времени t;

p – ковариационная матрица ошибок (мера точности оценивания вектора состояния) в момент времени t.

В самом начале, когда робот производит первые измерения, заполняется ковариационная матрица. Делается предположение, что эти элементы не взаимосвязаны. Это означает, что недиагональные элементы нулевые.

Тем не менее, когда робот начинает движение и производит новые измерения, местоположение робота и ориентиры начинают коррелировать. Соответственно, ковариационная матрица перестает быть разреженной.

Когда одометрия изменяется, новая позиция обновляется в расширенном фильтре калмана с использованием обновления одометрии.[5] Затем ориентиры извлекаются из окружающей среды, исходя из нового положения робота и

определяются их описания, необходимые для поиска соответствий. Затем робот пытается связать эти ориентиры с ориентирами, которые были раннее. Повторно наблюдаемые ориентиры используются для обновления позиции робота в фильтре. Ориентиры, которые не были замечены, будут добавлены в фильтр в качестве новых наблюдений. Принцип конечной работы метода представлен на рис. 4, где

1) Реальное положение робота
2) Треугольник показывающий работу алгоритма
3) Измерения положения от одометрии
4) Ориентиры

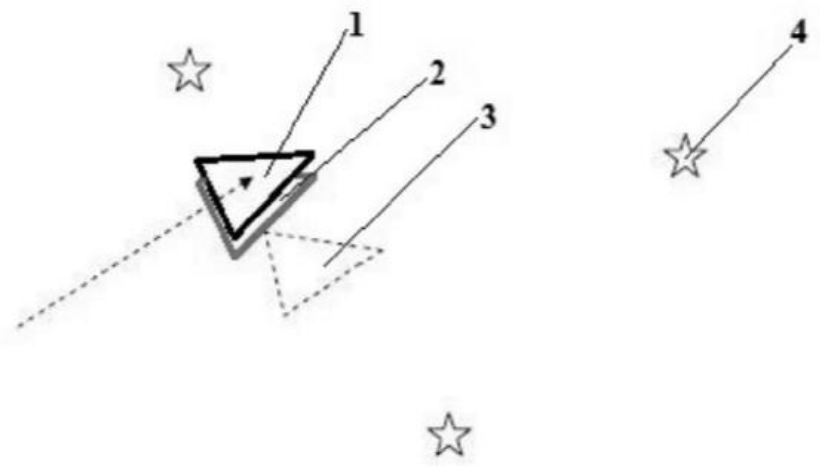

Рисунок 4 –Принцип работы мтода ekf-slam

Несмотря на все свои преимущества, ekf-slam имеет ограничения на количество используемых ориентиров в системе. Связанно это с тем, что при появлении новых элементов в наблюдении они добавляются в вектор состояния. Таким образом, шум ковариационной матрицы растет квадратично, что приводит к увеличению времени обработки. [6]

2.4.2 Описание метода основанного на графе (graph-based slam)

Методы основанные на графах методы slam основанные на использовании графов[7], являются одними из самых современных, подходом к задаче одновременной локализации и построения карты. Одним из самых популярных методов, основанных на графах, является graphslam. Предпосылкой к появлению этого метода была возможность представления задачи slam в виде разреженного графа и связей между его узлами. Узлы графа представляют собой положение робота и элементы карты. Относительные перемещения робота и положения элементов карты (относительно робота), выражаются на графе как связи между соответствующими узлами.

Существенное преимущество graph-based slam состоит в том, что по сравнению с фильтром калмана количество вычислений и памяти, которые требуются для обновления и сохранения ковариационной матрицы, не растет квадратично с числом элементов.[8] Главным недостатком методов slam основанных на графах, это требования больших затрат на вычисление, если робот прошел достаточно длинный путь

2.4.3 Описание метода основанного на фильтре частиц

Как говорилось выше, расширенный фильтра калмана, не является универсальным способом решения задачи slam.[9]. При данном подходе каждая частица характеризуется как возможное положение робота в данный момент времени. Будучи математическими моделями, эти фильтры описывают распределение вероятности в виде дискретного набора частиц в пространстве состояний, для задачи slam.

Первым шагом итерации локализации фильтра частиц является генерация нового частичного распределения для данной модели движения и примененного управления.

Следующим шагом, рассчитывается вес каждого значения. Частицам, для которых прогнозируемые значения соответствуют измеренным, даются большие веса.

И последний этап — это повторная выборка частиц. Она основывается на весах частиц из предыдущего распределения, которые берутся случайным образом, создавая новое распределения.

2.4.4 Выбор алгоритма реализации системы ориентации робокара

В результате анализа всех достоинств и недостатков существующих подходов к решению задачи ориентации выбран подход, основанный на фильтре частиц из-за его эффективности для мобильных роботов. Данный алгоритм хорошо себя показывает в закрытых помещениях, в которых отсутствуют перепады высот. Так же критерием выбора данного алгоритма стало наличие большого количества литературы по уже реализованным алгоритмам slam

2.5 Выбор платформы для работы с робокаром

Основной платформой для взаимодействия с робокаром была Robot Operating System из-за ее гибкости и удобности при работе с мобильными роботами [10]. ROS подходит для широкого спектра робототехнических устройств и отлично подходит для решения поставленных задач. Благодаря пакетам, встроенным в эту открытую робототехническую платформу, удобно устанавливать связь между датчиками робокара и собирать данные об его окружении. С точки зрения архитектуры ROS состоит из независимых узлов, которые называются нодами (node). Между ними идут потоки данных - топики (topics). Ноды внутри себя могут содержать всё, что угодно, главное - чтобы у него был определенный ROS API.

2.6 Выбор языка программирования

Для возможности создания программного модуля необходимо выбрать язык программирования. Среди широкого разнообразия языков программирования в данном проекте были рассмотрены только языки которые удобно использовать при работе с робототехническими устройствами. В данной выборке присутствуют только современные языки высокого уровня. Важным условием при выборе языка программирования является его поддержка ros. Сравниваемые языки программирования приведены в таблице 1.

Таблица 1 – Сравнение языков программирования

Язык	Преимущества	Недостатки
Java	• Подходит для разработки больших проектов • Возможность запуска программ на различных платформах	• Высокое потребление ресурсов • Язык довольно сложный для реализации больших проектов • Узкий функционал для работы с робототехническими датчиками
C++	• Высокая производительность • Поддержка значительного числа платформ • Большое число встроенных модулей	• Высокая сложность разработки
C#	• Бесплатное распространение • Открытый исходный код • Очень легковесный, удобно использовать даже на старых компьютерах	• Сложность для новичков
Python	• Простой и понятный синтаксис • Высокая скорость разработки • Большое количество библиотек, упрощающих написание кода • Поддержка работы ros	• Сложно маштабировать

Исходя из вышеперечисленных достоинств и недостатков каждого языка программирования делается выбор в пользу Python [11].

Python – современный язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью с большим количеством документации как на английском, так и на русском языках.

3. Техническое задание

3.1 Общие сведения

Полное наименование модуля и ее условное обозначение

Система картографирование и локализации робокара на местности

Наименование предприятий разработчика и заказчика модуля и их реквизиты

Разработчик: Ольхов Е. О., РК9-84Б.

Заказчик: кафедра РК9 МГТУ им. Н. Э. Баумана.

Юридический адрес: г. Москва, 2-я бауманская улица, 5 стр.1.

Плановые сроки начала и окончания работы по созданию системы

Начало работы:

Окончание работы:

3.2 Назначение и цели создания системы

Назначение разрабатываемой системы – обеспечение автономности транспортировки грузов на складском помещении с помощью автоматизированного робокара.

Модуль обеспечивает:

- Картографирование местности и определение окружающих объектов
- Точное определение нахождения робота на карте
- Корректное взаимодействие с технической базой робокара

3.3 Требования к системе в целом

Разработанный алгоритм должен соответствовать следующим критериям

- Стабильно распознавать объектов внутри складского помещения
- Для обмена данными с робокаром используется микрокомпьютер raspberry pi
- Разработанный исходный код программной части написан на языке python на базе открытой робототехнической платформы ros

Техническое задание разработано в соответствии с ГОСТ 34.602-89 [12].

4. Структурное проектирование

4.1. Выбор технической базы

Для проведения испытаний и проверки работы системы использовался робот с открытым персональным кодом turtlebot (рисунок 4). Данное устройство имеет всю необходимую техническую базу для реализации системы ориентации в пространстве.[13] Основным преимуществом выбранной платформы является гибкий выбор конфигурации turtlebot. При разработке проекта были учтены технические особенности робота компании denford. Аналогично беспилотному робокару turtlebot обладает встроенным лидаром и одноплатным компьютером raspberry pi. За основу была взята модель turtlebot3 из-за ее совместимости с версией ros 18.04 melodic, на базе которой был написан программный код. При реализации алгоритма были использованы пакеты turtlebot3_gazebo для запуска модели робота и корректной работы одометрических и оптических датчиков и turtlebot3_teleop для управления роботом с помощью клавиатуры.

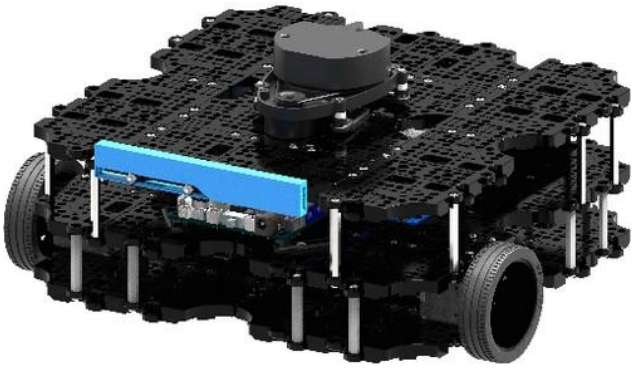

Рисунок 5 – Turtlebot

4.2. Представление карты местности

В качестве метода реализации алгоритма локализации был выбран фильтр частиц. Данный метод является одним из самых распространенных решений задачи локализации, зарекомендовав себя при использовании в мобильных роботах, выполняющих задачи в закрытых пространствах.

В большинстве алгоритмов локализации данные об окружающем пространстве хранится в так называемой карте-сетке (grid map), которая представляет собой матрицу, каждый элемент которой описывает прямоугольный участок пространства mij, где i – изменяется от 0 до ny и j – от 0 до nx.

в самой простой реализации карты-сетки [14], если элемент матрицы равен 1, то это означает, что в соответствующем участке находится объект, если элемент равен 0, то участок свободен. Данное представление сравнимо с бинарным растровым изображением.

В более сложной модели карта-сетка пространства включает в себя информацию о вероятности нахождения какого-либо объекта в каждой ячейку карты и, следовательно, каждому элементу присваивается значение от 0 до 1. Где 1 означает, что достоверное нахождение объекта помещения, соответствующая карта, а 0 отсутствие каких-либо объектов. Такой подход позволяет учесть статистические характеристики измерений, что, в свою очередь, повышает детализировалось получаемой карты. При этом сравнение скана с картой проводится путем сложения плотностей вероятности, содержащихся в ячейках карты, в которые попали точки текущего скана. В отличие от предыдущего варианта, здесь можно более точно оценить степень совпадения, и как следствие определить преобразование координат с меньшей погрешностью.

На рисунках 6 и 7 приведены графические интерпретации некоторого участка помещения в виде карты-сетки:

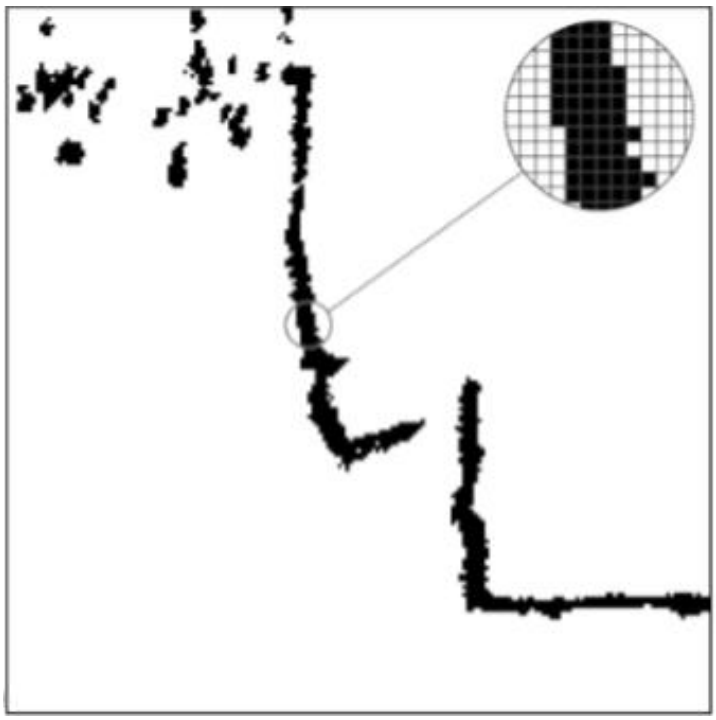

Рисунок 6 – Бинарое представление

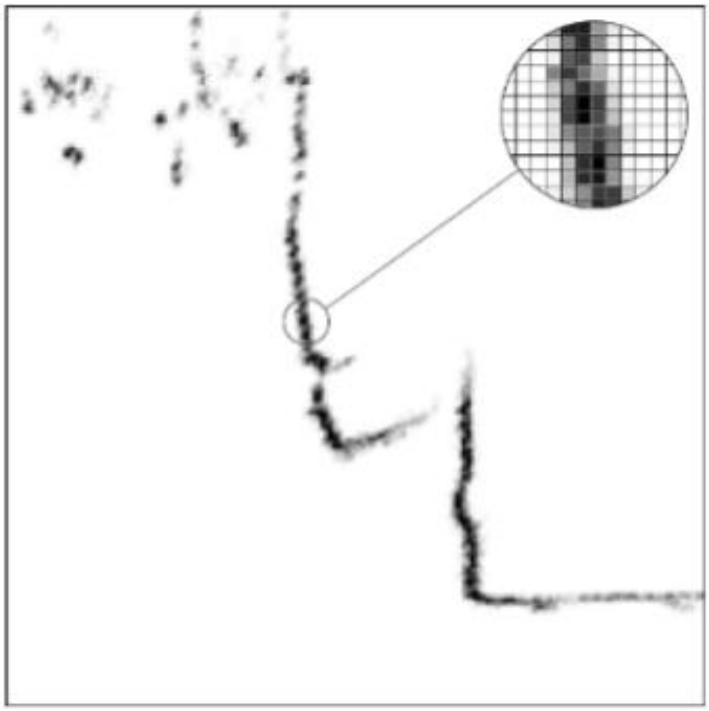

Рисунок 7 – Вероятностное представление

в данном подходе есть существенный недостаток, чтобы обеспечить высокую детализацию карты, необходимо выбрать минимальный размер

ячейки, сопоставимый с требуемой точностью локализации. Это естественно увеличивает объем информации для хранения карты.

4.3. Алгоритм

Фильтр частиц является одним из самых популярных методов оптимальной фильтрации.

Фильтр частиц позволяет получить оценку (приближенное значение) параметров системы или объекта (обозначим их как параметры а) которые нельзя измерить напрямую. Для построения этой оценки фильтр использует измерения других параметров связанных с первыми. Принцип работы фильтра частиц изображен на рисунке 8.

Рисунок 8

Для оценки параметров а фильтр создает множество гипотез (частиц) о текущем значении этих параметров. В начальный момент времени эти гипотезы абсолютно случайны, но на каждой итерации цикла фильтрации фильтр будет убирать гипотезы, которые не пройдет проверку достоверности.

Таким образом из множества гипотез в конце концов останутся только те, которые наиболее близки к истинному значению.

Алгоритм фильтра частиц разделим на две части: инициализация и основной цикл фильтрации.

Инициализация. Прежде чем приступать к фильтрации, фильтр частиц нужно инициализировать — задать параметры, начальное распределение и другие условия работы фильтра. Основной параметр фильтра частиц — число этих самых частиц. Чем больше частиц — тем точнее фильтр, и тем больше вычислений нужно проводить на каждой итерации основного цикла. Начальное распределение частиц зависит от априорной информации. Например, если нам известно, что робот находится в клетке с координатами (0, 0), то все частицы должны быть случайным образом распределены внутри этой клетки. А также ориентация каждой частицы должна быть равна 0.

Основной цикл фильтрации.

Основной цикл фильтрации разделен на три фазы:

- Движение (motion update)
- Измерение (measurement update)
- Отсев (resampling)

Движение. На этом этапе робот совершает движение и, так как движение робота происходит с погрешностями, теряет информацию о своем местоположении. Для большинства мобильных роботов этот этап соответствует расчету одометров. При каждой итерации алгоритма к текущему положению частицы прибавляется перемещение робокара полученное с одометрических датчиков

Измерение веса частиц. Каждая частица представляет собой возможное положение и ориентацию для мобильного робота

Предположим, что за первое перемещение робота, его одометры рассчитали его положение и ориентацию с некоторой ошибкой, тогда после остановки робота, мы производим опрос датчика и на основе полученных координат рассчитываем вес для каждой частицы . Для этого мы пользуемся данными поступающими нам с лидара. Сравнивая реальные данные о расстоянии до ближайших ориентиров с гипотетическими данными в каждой точке мы получаем отклонение для каждой гипотезы, которое характеризует вероятность достоверности частицы.

Отсев. На данном этапе происходит отсев частиц с маленьким весов, и выбирается частица с самым большим весов, координаты этой частицы и становится новыми координатами положения робота и расчет в следующей итерации по одометрам начнется от этих координат.

5. Рабочее проектирование

5.1. Построение и настройка карты

Построение карты происходит с помощью встроенного пакета ros map_server который обрабатывает и сохраняет карту из данных полученных с оптических датчиков. В начальный момент времени робот получает неполные данные об окружающем пространстве. На рисунках 10,11,12 изображен процесс построения карты. Поэтапно перемещая робокар карта становится более точной и в итоге мы получаем цельную карту помещения.

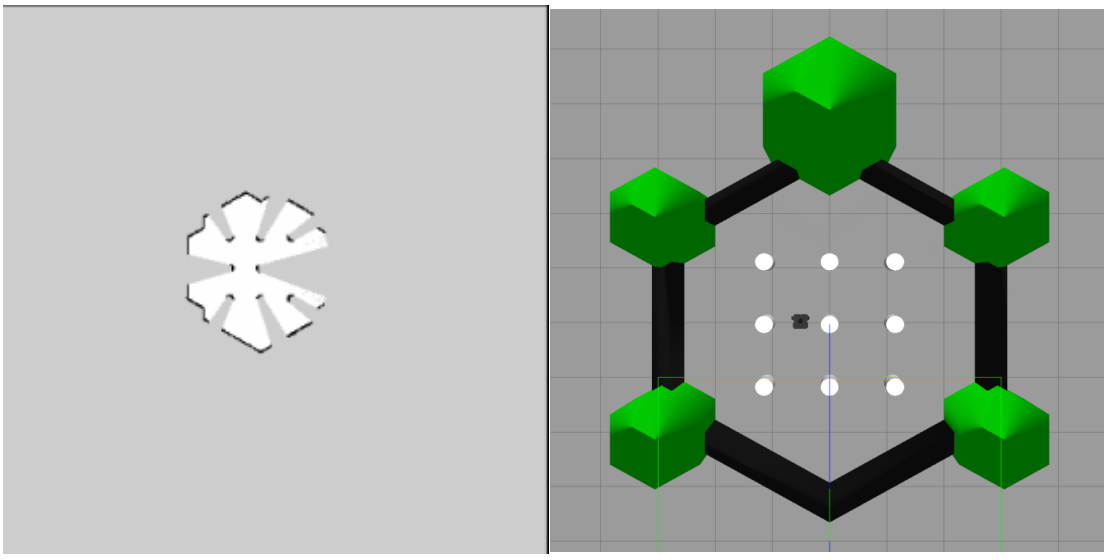

Рисунок 9 – Карта построенная в начальный момент времени

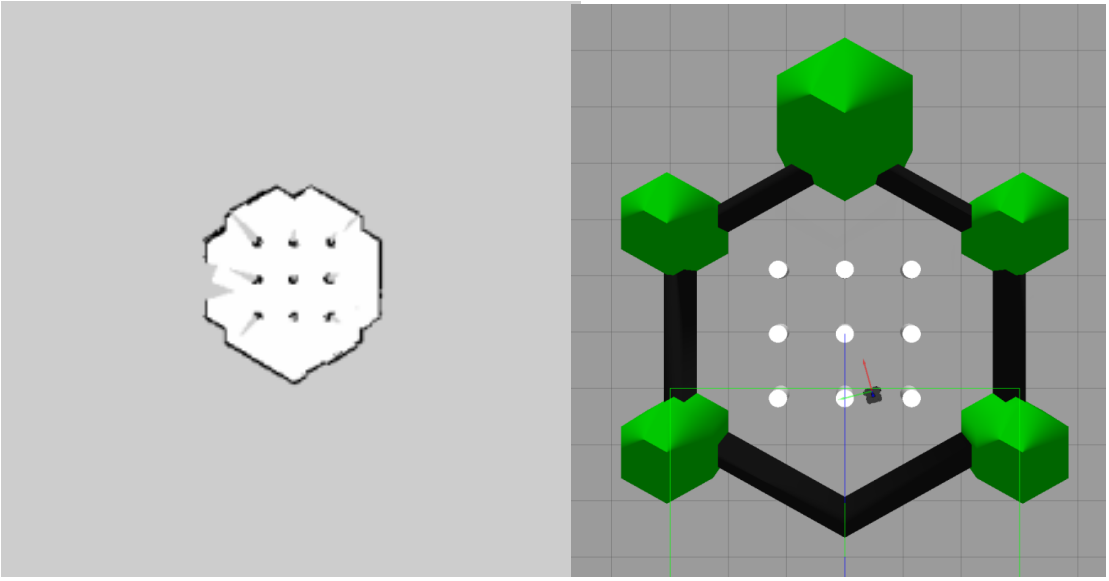

Рисунок 10 – Карта после одного перемещения

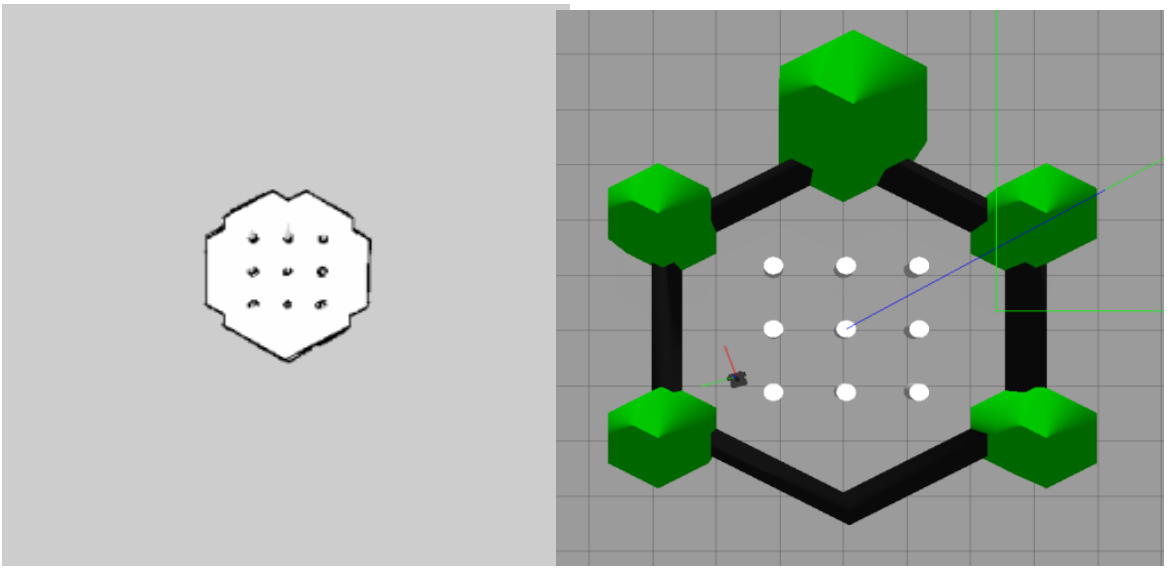

Рисунок 11 – Карта полученная после исследования местности

В реализованном алгоритме использовалось бинарное представление окружающей местности. На вход поступает файл в формате yaml, который хранит в себе метаданные карты и указывает на изображение карты . Обработка изображения происходит с помощью встроенного пакета ros map_server[15], который позволяет обработать изображение и построить карту-сетку. Полученное на вход изображение описывает состояние каждой точки карты цветом пикселя. Более белые пиксели заняты, а более черные пиксели свободны. На рисунке 13 представлена карта использовавшаяся мной для проверки точности работы алгоритма. При выборе местности я руководствовался несколькими критериями.

1) Карта должна быть приближена к реальному складскому помещению и представлять собой замкнутое пространство с узкими коридорами
2) Территория должна иметь повторяющийся паттерн для усложнения задачи локализации алгоритмом

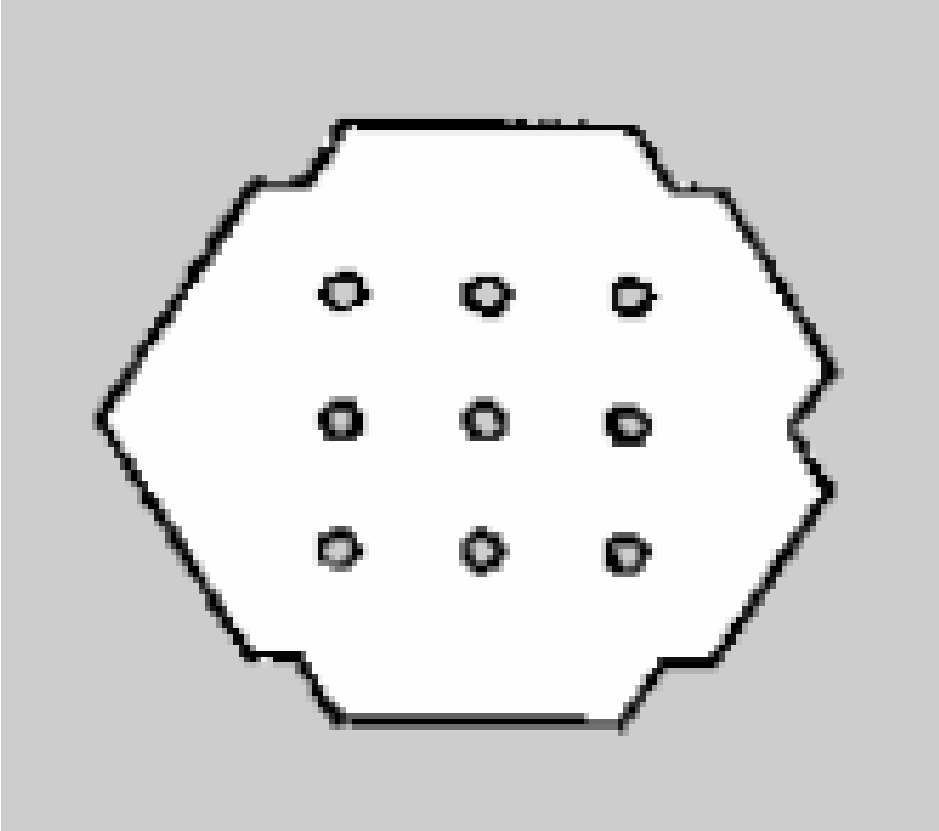

Рисунок 12 – Итоговая карта помещения

5.2. Разработка модуля ориентации робокара

Разработка модуля производится на языке питон на базе открытой робототехнической платформы ROS. Python – Высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью. При разработке модуля ориентации уделяется тщательное внимание стабильному обмену данных между роботом, одометрическими датчиками и лидаром. На базе встроенных пакетов /rplidar и

/odom программа получает данные от робота. Программный код приведен внутри листинга 1.

```
    class Localizator:
        def __init__(self):
            self.scan_msg = None
            self.odom_msg = None
            rospy.init_node('particle_filter',
anonymous=True)
            self.rate = rospy.Rate(25) # 25hz
            self.odom_sub   =   rospy.Subscriber("odom",
Odometry, self.odom_callback)
            self.laser_sub = rospy.Subscriber("scan",
LaserScan, self.laser_callback)
```

Листинг 1 — Получение данных с датчиков

Цикл фильтрации алгоритма был реализован в трех основных функциях внутри основного класса particle_filter().

Move_base() – движение

Particle_likelihood() – измерение

Resample_particles() – отсев

Первым этапом цикла был разработан алгоритм перемещения точек move_base, который с помощью данных с одометричесих датчиков и пакета /odom рассчитывает новые значения положения всех частиц на карте. Функция вычисляет новое положение частицы на карте, вычитая из актуальных данных одометрии данные с предыдущего этапа.

На Реализация данной функции приведена внутри листинга 2

```
        def move_particles(self):
        remove_indices = []
            if self.odom_msg is not None:
                if self.last_odom is not None:
                    for                 i                 in
```

```
range(self.num_of_particles):
                        new_width                    =
self.particles[i].get_width() + \

self.odom_msg.pose.pose.position.x      *      20      -
self.last_odom.pose.pose.position.x * 20

self.particles[i].set_width(new_width)
                        new_height                   =
self.particles[i].get_height() - \

self.odom_msg.pose.pose.position.y      *      20      +
self.last_odom.pose.pose.position.y * 20

self.particles[i].set_height(new_height)
                new_width *= self.meter_to_pixel
                new_height *= self.meter_to_pixel
                self.last_odom = self.odom_msg
```

Листинг 2 — Функция перемещения

В реализации алгоритма локализации был использован вспомогательный модуль OccupacityGrid.py внутри которого мной была написана функция для подсчета кротчайшего расстояния от заданной точки до ближайшей стены. В работе данного модуля используется пакет языка python для векторизации изображения с помощью которого мы обрабатываем полученную карту. Для подсчета веса частиц используется функция Particle_likelihood() внутри основного класса программы. Получая данные с лидара в каждой из частиц мы можем построить гипотезу относительно того, в какой точке находится ближайшее препятствие на карте, например это может быть стенка складского шкафа или другой робот. Функция из модуля OccupacityGrid.py определяет ошибку для каждой частицы. На данном этапе алгоритма мы используем эту

ошибку в квадрате чтобы задать более высокий вес частицам с меньшим отклонением. Реализация функции измерения приводится ниже в листинге 3

```
        def particle_likelihood(self, particle):
         tot_prob = 0
            if self.scan_msg is not None:
             if self.odom_msg is not None:
                q = self.odom_msg.pose.pose.orientation
                quaternion = [q.x, q.y, q.z, q.w]
                roll,pitch,yaw                          =
euler_from_quaternion(quaternion)
                k = 0
                for                 i                 in
range(len(self.scan_msg.ranges)):
                        Range = self.scan_msg.ranges[i]
                        pixel_angle     =     yaw     +
self.scan_msg.angle_min + self.scan_msg.angle_increment *
i
                        wall_x = (particle.get_height())
-      Range      *      self.map1.info.resolution      *
math.cos(pixel_angle)
                        wall_y  =  (particle.get_width())
-      Range      *      self.map1.info.resolution      *
math.sin(pixel_angle)
                        d                               =
self.occupancy_field.get_closest_obstacle_distance(wall_x
,wall_y)
                        tot_prob += math.exp(-d**2)
                        k += 1
                tot_prob = tot_prob/k
```

```
                return tot_prob
```

Листинг 3 — Функция измерения

С помощью функции compute_weight каждой частице весовое значение, по которому будет производиться отсев. Программный код данной функции приводится ниже в листинге 4

```
        def compute_weights(self):
            self.sum_of_weights = 0.0
        self.particle_weights = []
            for particle in self.particles:
            particle_weight                                =
self.particle_likelihood(particle)

self.particle_weights.append(particle_weight)
                self.sum_of_weights += particle_weight
```

Листинг 4 — Функция задания веса для частиц

Функция отсева обрабатывает массив весов всех частиц и выбирает из них гипотезы, которые являются наиболее достоверными. Отсев частиц происходит исходя из значения их веса, чем выше вес частицы тем выше шанс ее попадания в новое поле гипотез. Таким образом на данном этапе мы избавляемся от наименее достоверных предположений о нахождении робота. Программный код данной функции приводится ниже в листинге 5

```
            def resample_particles(self):
        if self.sum_of_weights == 0:
            return
            new_particles = []
        s = sum(self.particle_weights)
        cum_sum                                            =
(np.cumsum(self.particle_weights).astype(np.float))/s
```

```
        pos = ((np.arange(self.num_of_particles) +
np.random.random())/self.num_of_particles)
        indexes = np.zeros(self.num_of_particles, "i")
        i = 0
        j = 0
        while i < self.num_of_particles and j <
self.num_of_particles:
            if pos[i] < cum_sum[j]:
                indexes[i] = j
            i += 1
            else:
            j += 1)
        for k in range(self.num_of_particles):
            if k in indexes:
            for i in range(np.sum(indexes == k)):
                particle = self.particles[k]
                particle.set_width(particle.get_width()
+ (np.random.random() - 0.5)/5)

particle.set_height(particle.get_height()              +
(np.random.random() - 0.5)/5)

new_particles.append(deepcopy(particle))
            self.particles = new_particles
        self.num_of_particles = len(new_particles)
```

Листинг 5 — Функция отсева

6. Апробирование

6.1. Тестирование работы алгоритма

Тестирование алгоритма проводилось на базе открытой робототехнической платформы ros. Была введена система координат для сравнения положения робота с положением, которое предсказывает алгоритм. В настойках алгоритма использовались разные значения переменной общего количества частиц n. Было принято взять 50,100,200,300,500. В приведенных ниже слайдах демонстрируется работа алгоритма при n = 100.

Для испытания работы алгоритма было проведено 5 экспериментальных запусков, начальное положение в каждом эксперименте было выбрано случайно. Результатом действия алгоритма было принято считать частицу с наибольшим весом. Для проверки качества полученных данных производилось сравнение координат частицы с реальным положением робота на карте. В начальным момент времени алгоритм не способен точно локализовать робота и предоставляет несколько гипотез. На рисунках 14 и 15 изображены начальное положение робота и отображение действий алгоритма соответственно.

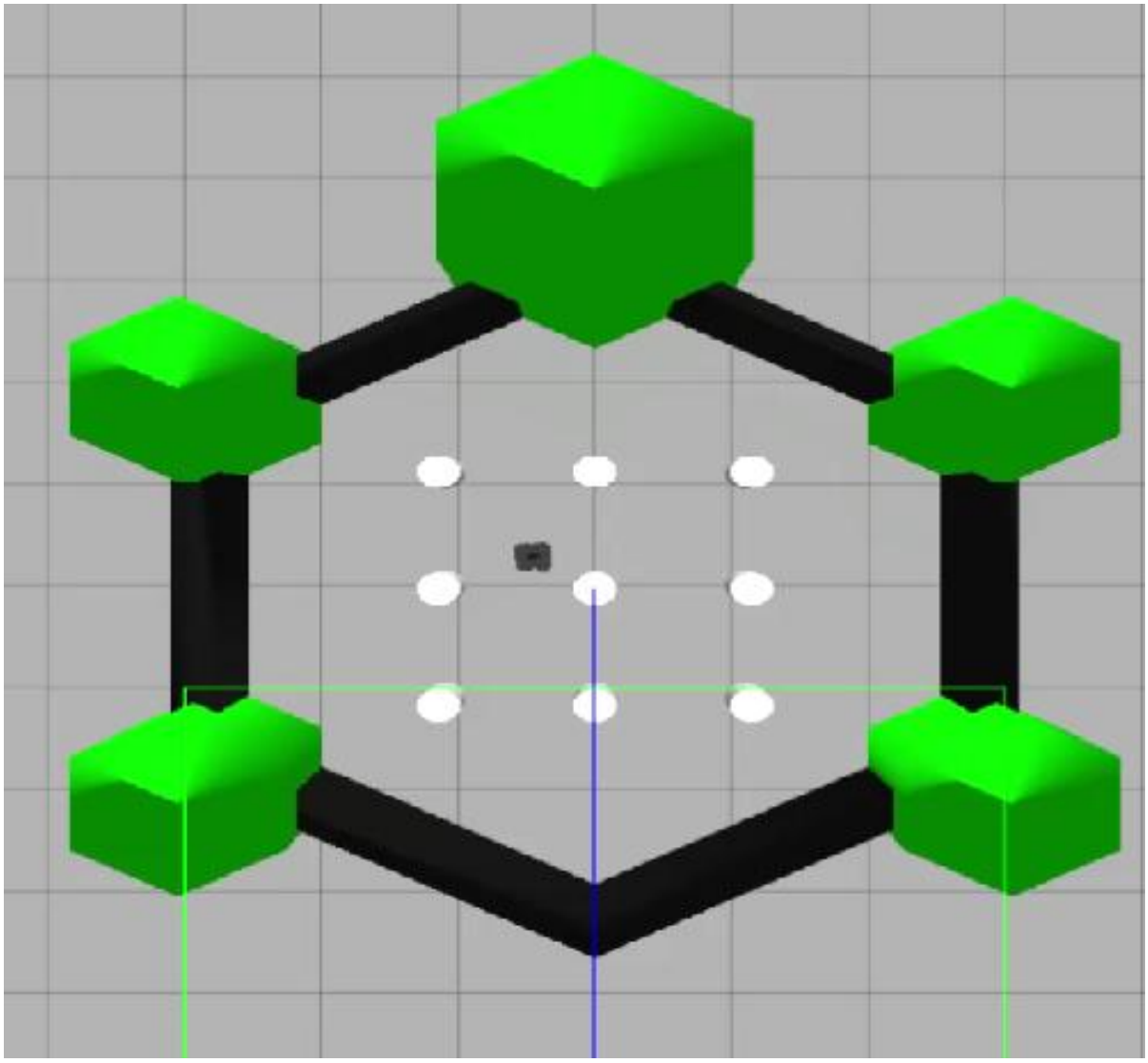

Рисунок 13 – Положение робота в начальный момент времени

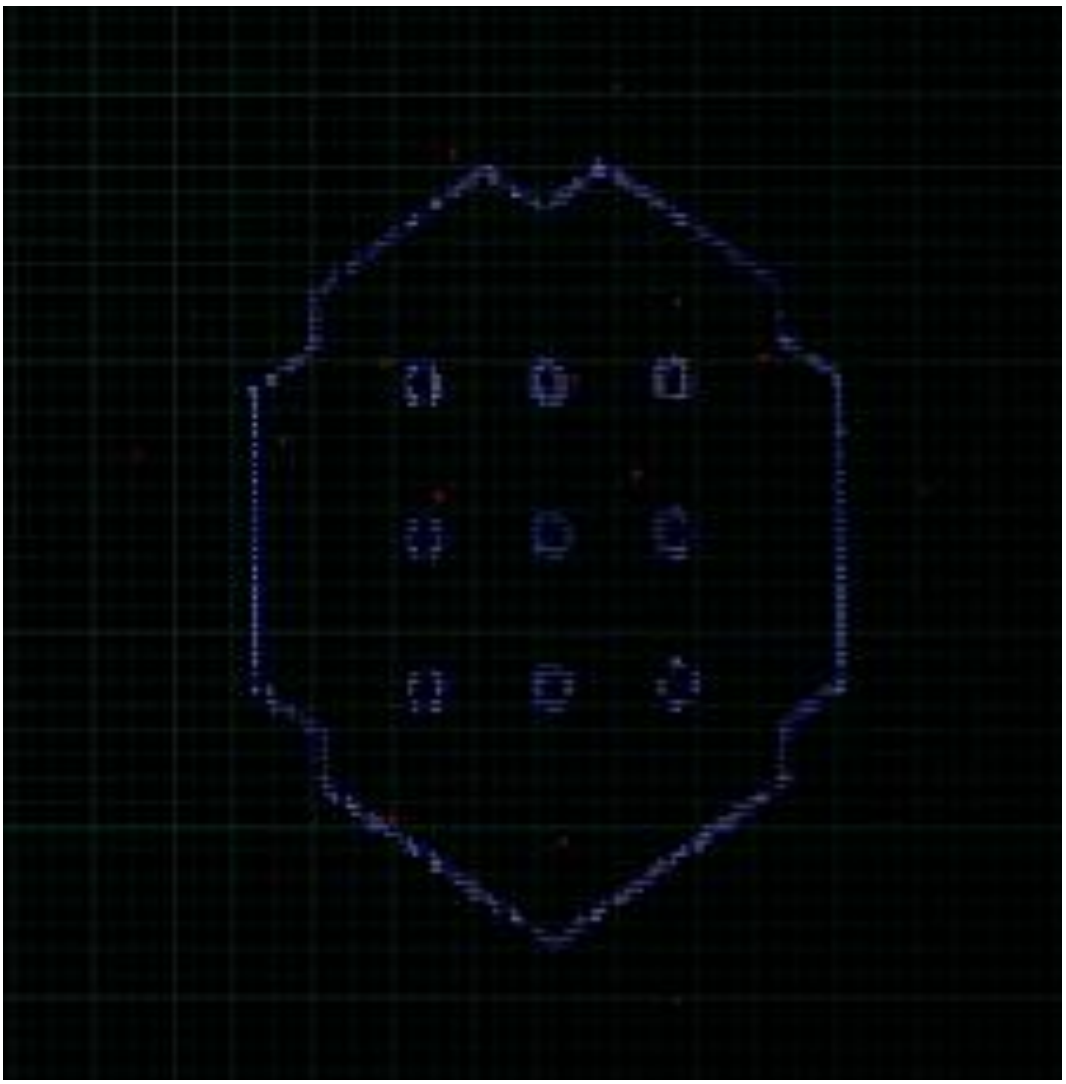

Рисунок 14 – Отображение действия алгоритма

По мере движения робокара по помещению алгоритм отсеивает большинство недостоверных гипотез и оставляет гипотезы с большим весом. На рисунке 16 изображено положение робота спустя 5 секунд движения. С помощью визуализации на рисунке 17 становится видно, что алгоритм оставляет несколько гипотез.

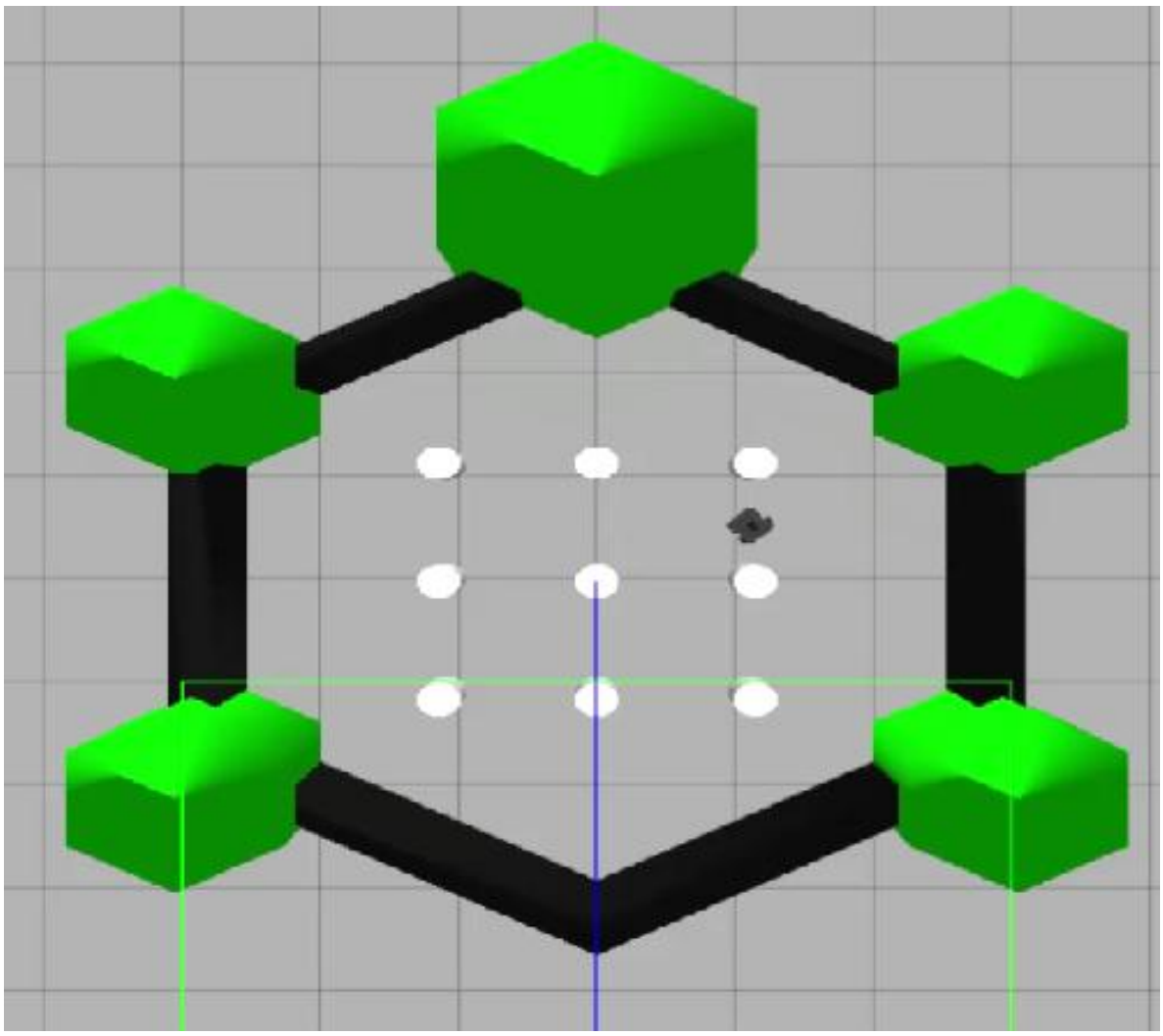

Рисунок 15 – Положение робота спустя 5 секунд

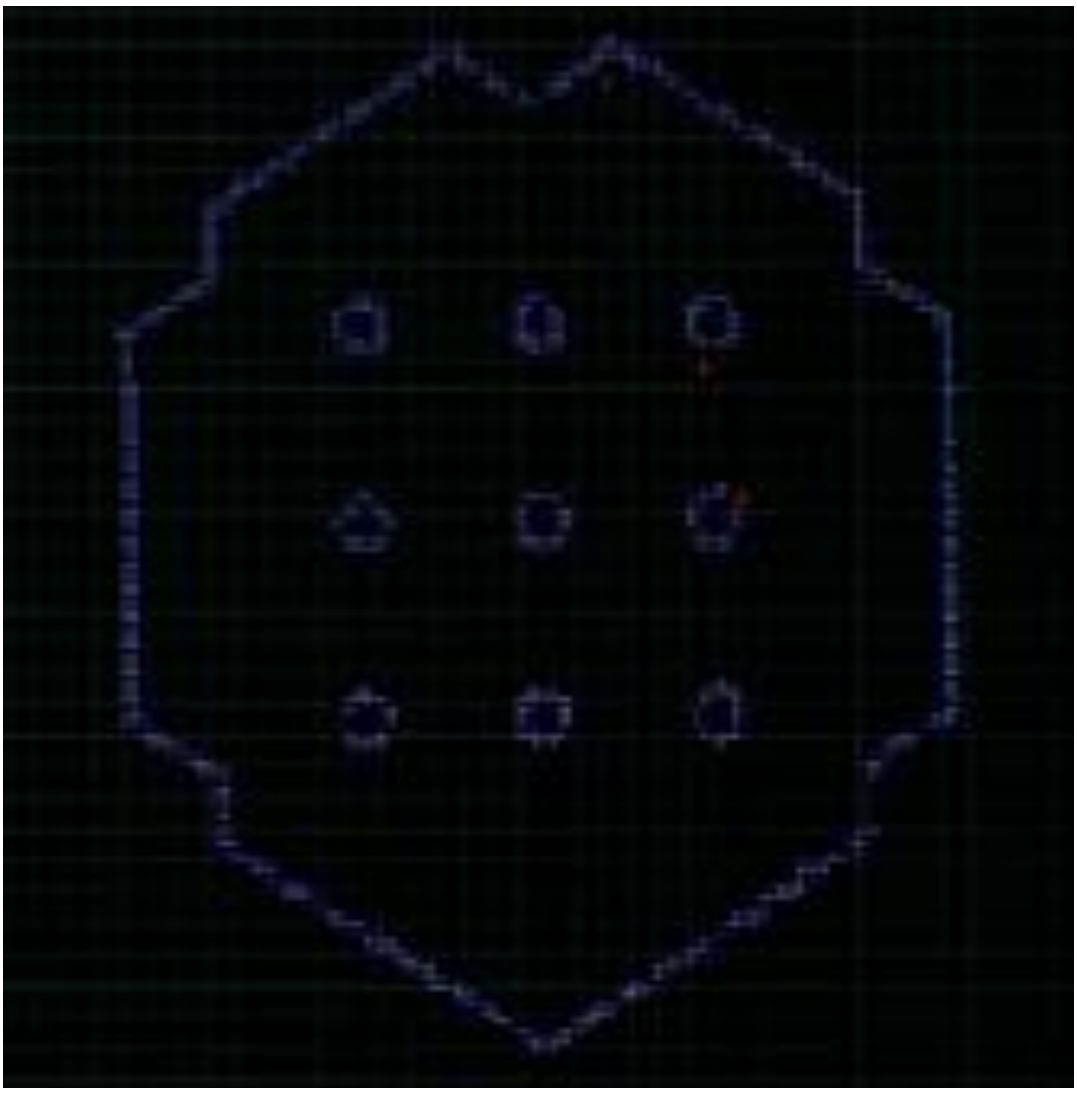

Рисунок 16 – Отображение действия алгоритма спустя 5 секунд

Спустя 15 секунд движения частицы сходятся к точке, которая предсказывает положение робота. Для того чтобы удостовериться в корректности работы алгоритма были произведены сравнения реального положения робота, которое было получено с помощью пакета gazebo открытой робототехнической платформы ros, с положением частицы с наибольшим весом. На рисунке 18 изображено конечное положение робота. Спустя 15 секунд движения все гипотезы сходятся к одному месту на карте, как изображено на рисунке 19.

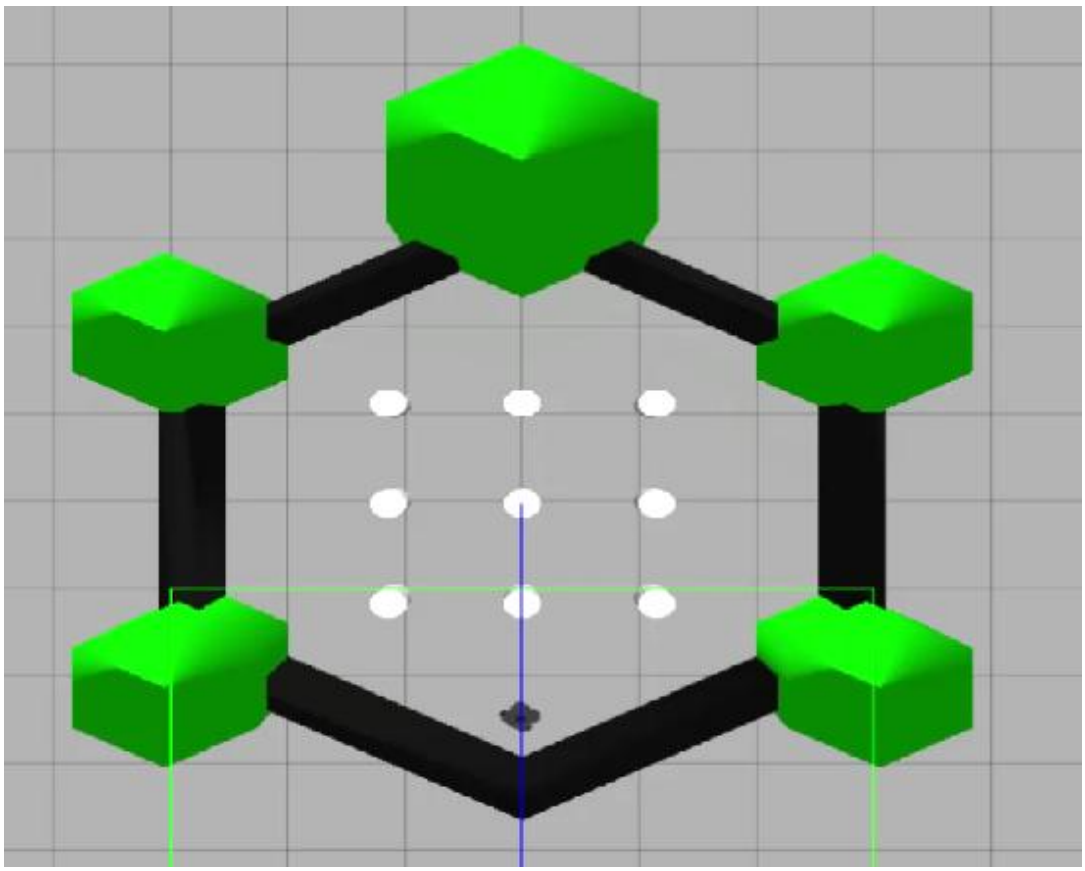

Рисунок 17 – Положение робота спустя 15 секунд

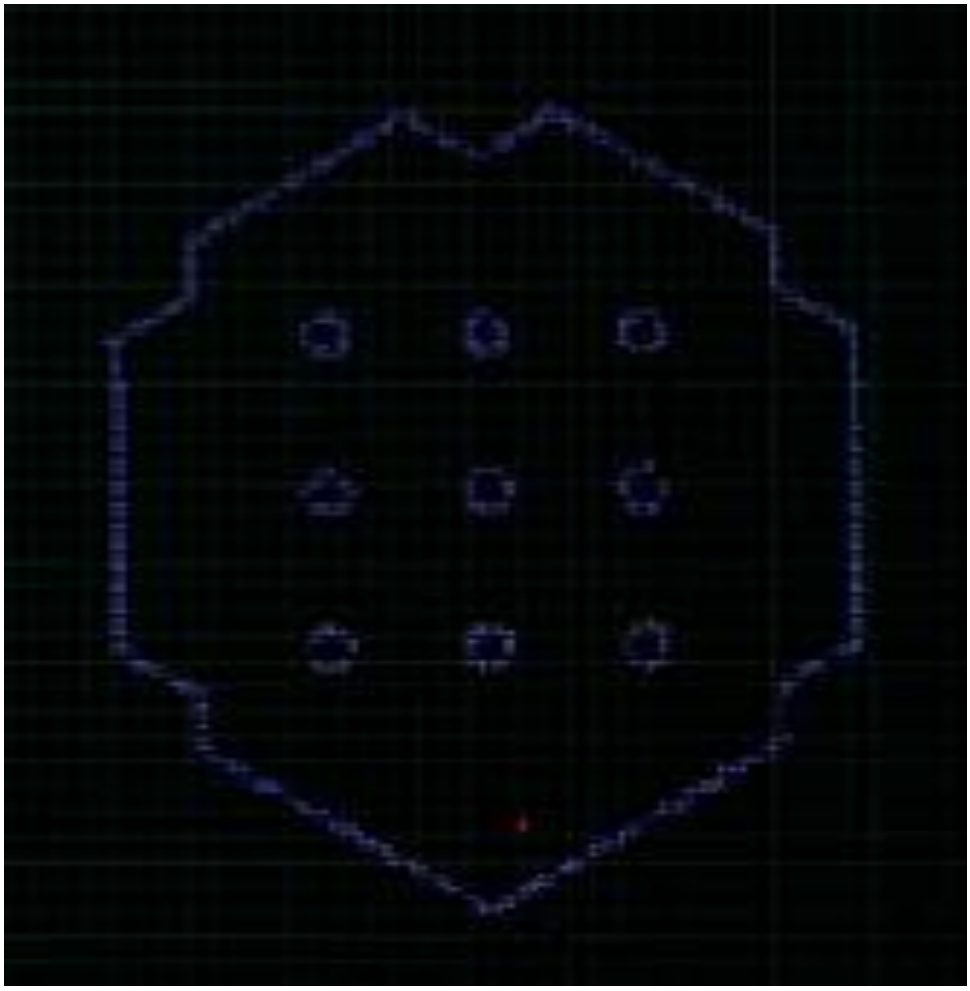

Рисунок 18 – Отображение действия алгоритма спустя 15 секунд

В результате тестирования нам удалость проверить работу модуля и доказать его работоспособность на практике. Реализованная нода точно локализует робота и позволяет в каждый момент времени определять положение робокара на складе. Результаты точности определения позиции робота на карте после 5 запусков показали, что алгоритм определяет текущую точку с погрешностью не более 1.12%. Результаты апробирования приведены в таблице 2, где X,Y – реальные координаты робота, x,y – координаты предсказанные алгоритмом.

Таблица 2 – Результаты апробирования

		5 с	10 с	15 с
(X,Y)	50	(121, 243)	(254, 256)	(82, 112)
(x,y)	50	(142, 267)	(267, 259)	(87, 114)
(X,Y)	100	(351,240)	(364,392)	(85,336)
(x,y)	100	(359,232)	(368,388)	(85,340)
(X,Y)	200	(277,502)	(241,488)	(359,232)
(x,y)	200	(260,536)	(220,232)	(359,232)
(X,Y)	300	(144, 442)	(121,334)	(170,201)
(x,y)	300	(166, 494)	(139,354)	(166,208)
(X,Y)	500	(335,340)	(337,409)	(256,379)
(x,y)	500	(355,348)	(332,404)	(256,371)

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта были выполнены следующие задачи:

1) Проведено исследование технического устройства робокар
2) Сформированы требования к решению проблемы картографирования и локализации
3) Осуществлен анализ технического решения и предложены варианты по технологической доработке устройства робокара
4) Рассмотрены актуальные варианты реализации картографирования, локализации территории и SLAM алгоритмы
5) Разработан код программной части на языке python на базе открытой робототехнической платформы ros;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Овсянников М.В., Буханов С. А., Подкопаев С. А. Организация индустриального интернета на основе SAP HANA CLOUD PLATFORM // М.: Издательство МГТУ им. Н. Э. Баумана. – 2019. – С. 6-10.
2) Getting-started-with-lidar // dronebotworkshop.com URL: https://dronebotworkshop.com/getting-started-with-lidar/
3) Enhanced SLAM for a Mobile Robot using Extended Kalman Filter / K.S. Choi, and S.J. Lee // International Journal of Precision Engineering and Manufacturing. – 2010. – №2. – С. 255–264.
4) О некоторых особенностях применения не доопределённых моделей в робототехнике / В.Э. Карпов // V Международная научно-практическая конференция «Интегрированные модели и мягкие вычисления в искусственном интеллекте». – М., – 2009. – Физматлит. – С. 520-532
5) Адаптация фильтра Калмана для использования с локальной и глобальной системами навигации / А.Н. Забегаев, В.Е. Павловский // XII национальная конференция по искусственному интеллекту с международным участием. – М., – 2010. – Физматлит. – С. 399-404
6) A Flexible and Scalable SLAM System with Full 3D Motion Estimation / S. Kohlbrecher, O. von Stryk, J. Meyer, U. Klingauf. // Safety, Security, and RescueRobotics (SSRR), 2011 IEEE International Symposium. – M., – 2011. – P. 155-160
7) An evaluation of 2d slam techniques available in robot operating system. / J. M. Santos, D. Portugal, and R. P. Rocha // In 2013 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR). – M., – 2013. – C. 1-6.

8) FastSLAM 2.0: An Improved Particle Filtering Algorithm for Simultaneous Localization and Mapping that Provably Converges / M. Montemerlo, S. Thrun /
9) .DP-SLAM: Fast, Robust Simultaneous Localization and Mapping Without Predetermined Landmark / A. Eliazar, R. Parr // Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence. – M., – 2003. – C. 345– 353.
10) An EKF SLAM Algorithm for Mobile Robot with Sensor Bias Estimation / X. Xie, Y. Yu, X. Lin, C. Sun // 2016 IEEE International Conference. – M., – 2016. – C. 281 – 285
11) Златопольский Д.М. Основы программирования на языке Python. – М. ДМК Пресс, 2017.
12) ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы.
13) TurtleBot // wiki.ros.org URL: http://wiki.ros.org/Robots/TurtleBot (дата обращения: 14.05.2022).
14) Построение карты мобильным роботом, оснащенным лазерным дальномером, методом рекуррентной фильтрации / С.Л. Зенкевич, А.А. Минин // Мехатроника, автоматизация, управление. – М., – 2007. – № 8. – С. 5-12
15) map_server.html // library.isr.ist.utl.pt URL: http://library.isr.ist.utl.pt/docs/roswiki/map_server.html

Приложение А

Программный код модуля локализации

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

import rospy
import numpy as np
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import Odometry
import message_filters
from particle import Particle
from animation import mapDrawer
from Tkinter import *
from scipy import misc
from copy import deepcopy
from tf.transformations import euler_from_quaternion
from random import randrange

from nav_msgs.srv import GetMap
from occupancy_field import OccupancyField
import math

class Localizator:
    def __init__(self):
    self.map_width = 639
        self.map_height = 810
        self.map_path                                    =
"/home/egor/catkin_ws/src/navigation/scripts/slam_map2.pn
g"
        self.map = misc.imread(self.map_path)
        self.particles = []
    self.mutex = 0
        self.sum_of_weights = 0.0
        self.particle_weights = []
        self.num_of_particles = 100
        self.scan_msg = None
        self.odom_msg = None
        self.last_odom = None
        self.meter_to_pixel = 30.8
        self.window = Tk()
        self.map_drawer = mapDrawer(self.window)
```

```
            rospy.init_node('particle_filter',
anonymous=True)
            self.rate = rospy.Rate(25) # 25hz
            self.odom_sub    =    rospy.Subscriber("odom",
Odometry, self.odom_callback)
            self.laser_sub   =   rospy.Subscriber("scan",
LaserScan, self.laser_callback)

            self.map_frame = "map"
            self.map_server                              =
rospy.ServiceProxy('static_map', GetMap)
            self.map1 = self.map_server().map
            self.occupancy_field                         =
OccupancyField(self.map1)

        def move_particles(self):
        remove_indices = []
            if self.odom_msg is not None:
                if self.last_odom is not None:
                    for               i               in
range(self.num_of_particles):
                        new_width                        =
self.particles[i].get_width() + \

self.odom_msg.pose.pose.position.x      *      20       -
self.last_odom.pose.pose.position.x * 20

self.particles[i].set_width(new_width)
                        new_height                       =
self.particles[i].get_height() - \

self.odom_msg.pose.pose.position.y      *      20       +
self.last_odom.pose.pose.position.y * 20

self.particles[i].set_height(new_height)
                new_width *= self.meter_to_pixel
                new_height *= self.meter_to_pixel
                self.last_odom = self.odom_msg
        cnt = 0

        def draw_map(self):
        for dot in self.occupancy_field.get_dots():
            self.map_drawer.draw_dot_blue(int(dot[0]+10)
,int(dot[1]+10))
```

```
        def particle_likelihood(self, particle):
        tot_prob = 0
            if self.scan_msg is not None:
             if self.odom_msg is not None:
                 q = self.odom_msg.pose.pose.orientation
                 quaternion = [q.x, q.y, q.z, q.w]
                 roll,pitch,yaw                          =
euler_from_quaternion(quaternion)
                 k = 0
                 for                 i                 in
range(len(self.scan_msg.ranges)):
                         Range = self.scan_msg.ranges[i]
                         pixel_angle     =     yaw     +
self.scan_msg.angle_min + self.scan_msg.angle_increment *
i
                         wall_x = (particle.get_height())
-      Range      *      self.map1.info.resolution      *
math.cos(pixel_angle)
                         wall_y  =  (particle.get_width())
-      Range      *      self.map1.info.resolution      *
math.sin(pixel_angle)
                         d                               =
self.occupancy_field.get_closest_obstacle_distance(wall_x
,wall_y)
                         tot_prob += math.exp(-d**2)
                         k += 1
                 tot_prob = tot_prob/k
                 return tot_prob


        def sample_particles(self):
            for i in range(self.num_of_particles):
                while True:
                    particle_width                       =
int(np.random.uniform(0, self.map1.info.width))
                    particle_height                      =
int(np.random.uniform(0, self.map1.info.height))
                    if
np.any(self.map[particle_height][particle_width] != 255):
                        break
                particle    =    Particle(particle_width,
particle_height)
                self.particles.append(particle)

        def resample_particles(self):
```

```
        if self.sum_of_weights == 0:
            return
            new_particles = []
        s = sum(self.particle_weights)
        cum_sum                                          =
(np.cumsum(self.particle_weights).astype(np.float))/s
        pos   =   ((np.arange(self.num_of_particles)   +
np.random.random())/self.num_of_particles)
        indexes = np.zeros(self.num_of_particles, "i")
        i = 0
        j = 0
        while   i   <   self.num_of_particles   and   j   <
self.num_of_particles:
            if pos[i] < cum_sum[j]:
                indexes[i] = j
            i += 1
            else:
            j += 1)
        for k in range(self.num_of_particles):
            if k in indexes:
            for i in range(np.sum(indexes == k)):
                particle = self.particles[k]
                particle.set_width(particle.get_width()
+ (np.random.random() - 0.5)/5)

particle.set_height(particle.get_height()               +
(np.random.random() - 0.5)/5)
                new_particles.append(deepcopy(particle))
            self.particles = new_particles
        self.num_of_particles = len(new_particles)

        def compute_weights(self):
            self.sum_of_weights = 0.0
        self.particle_weights = []
            for particle in self.particles:
            particle_weight                              =
self.particle_likelihood(particle)

self.particle_weights.append(particle_weight)
                self.sum_of_weights += particle_weight

        def odom_callback(self, odom):
        if self.mutex == 0:
            self.odom_msg = odom
```

```
        def laser_callback(self, scan):
            if self.mutex == 0:
             self.scan_msg = scan

        def particle_filter(self):
            self.sample_particles()
        self.draw_map()
        self.map_drawer.draw_particles(self.particles)
            while not rospy.is_shutdown():
                self.compute_weights()
                self.resample_particles()

self.map_drawer.draw_particles(self.particles)
                self.window.update()
            for i in range(0,10):
             self.move_particles()

    self.map_drawer.draw_particles(self.particles)
             self.window.update()
                 self.rate.sleep()

    if __name__ == '__main__':
        try:
            loc = Localizator()
            loc.particle_filter()
        except rospy.ROSInterruptException:
            pass
```

Приложение Б

Программный код вспомогательного модуля для измерения веса частицы

```
import rospy

from std_msgs.msg import Header, String
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import PoseStamped, PoseWithCovarianceStamped, PoseArray, Pose, Point, Quaternion
from nav_msgs.srv import GetMap
from copy import deepcopy

import tf
from tf import TransformListener
from tf import TransformBroadcaster
from tf.transformations import euler_from_quaternion, rotation_matrix, quaternion_from_matrix
from random import gauss

import math
import time

import numpy as np
from numpy.random import random_sample
from sklearn.neighbors import NearestNeighbors


class OccupancyField(object):

    def __init__(self, map):
        self.map = map
        X = np.zeros((self.map.info.width*self.map.info.height,2))

        total_occupied = 0
        curr = 0
        for i in range(self.map.info.width):
            for j in range(self.map.info.height):
                ind = i + j*self.map.info.width
                if self.map.data[ind] > 0:
                    total_occupied += 1
                X[curr,0] = float(i)
```

```
                X[curr,1] = float(j)
                curr += 1

        O = np.zeros((total_occupied,2))
        curr = 0
        for i in range(self.map.info.width):
            for j in range(self.map.info.height):
                ind = i + j*self.map.info.width
                if self.map.data[ind] > 0:
                    O[curr,0] = float(i)
                    O[curr,1] = float(j)
                    curr += 1

        nbrs                                        =
NearestNeighbors(n_neighbors=1,algorithm="ball_tree").fit
(O)
        distances, indices = nbrs.kneighbors(X)

        self.closest_occ = {}
        curr = 0
        for i in range(self.map.info.width):
            for j in range(self.map.info.height):
                ind = i + j*self.map.info.width
                self.closest_occ[ind]               =
distances[curr][0]*self.map.info.resolution
                curr += 1

    self.G = O

    def get_closest_obstacle_distance(self,x,y):
        x_coord          =          int(x          -
self.map.info.origin.position.x)
        y_coord          =          int(y          -
self.map.info.origin.position.y)
        if x_coord > self.map.info.width or x_coord <
0:
        return "1"
            return float('nan')
        if y_coord > self.map.info.height or y_coord
< 0:
        return "2"
            return float('nan')

        ind = x_coord + y_coord*self.map.info.width
```

```
            if                    ind                    >=
self.map.info.width*self.map.info.height or ind < 0:
            return "3"
                return float('nan')
            return self.closest_occ[ind]

        def get_dots(self):
        return self.G
```

Приложение В

Программный код код класса частицы

```
class Particle:
    def __init__(self, width = 0, height = 0):
        self.width = width
        self.height = height

    def get_width(self):
        return self.width

    def get_height(self):
        return self.height

    def get_cnt(self):
        return self.cnt

    def set_width(self, width):
        self.width = width

    def set_height(self, height):
        self.height = height
```

Приложение Г

Программный код модуля визуализации

```
from Tkinter import Tk, Canvas, PhotoImage, mainloop
from math import sin
from scipy import misc
import numpy as np
from particle import Particle


class mapDrawer:

    def __init__(self, window):
    self.window = window
    self.img_location =
"/home/egor/catkin_ws/src/navigation/scripts/slam_map2.pn
g"
    self.img = PhotoImage(file=self.img_location)
    self.old_img = misc.imread(self.img_location)
    self.WIDTH_OF_IMAGE = self.img.width()
    self.HEIGHT_OF_IMAGE = self.img.height()
    self.WIDTH_OF_IMAGE = 800
    self.HEIGHT_OF_IMAGE = 800
    self.number_of_particles =
np.zeros(shape=(self.HEIGHT_OF_IMAGE,
self.WIDTH_OF_IMAGE))
    self.canvas = Canvas(window,
width=self.WIDTH_OF_IMAGE*2,
height=self.HEIGHT_OF_IMAGE*2, bg="#000000")
    self.canvas.pack()
    self.canvas.create_image((self.WIDTH_OF_IMAGE/2,
self.HEIGHT_OF_IMAGE/2), image=self.img, state="normal")#
    self.old_red_pixels = []
    def clamp(self, x):
        return max(0, min(x, 255))

    def convert_to_six_digit_code(self, color):
        r, g, b = color
        return
"#{0:02x}{1:02x}{2:02x}".format(self.clamp(r),
self.clamp(g), self.clamp(b))

    def restore_old_image(self):
```

```
        for position in self.old_red_pixels:
        width, height = position

self.img.put(self.convert_to_six_digit_code(self.old_img[
height][width]), (width, height))
    self.old_red_pixels = []


    def draw_pos(self, particles):
    self.restore_old_image()
    for i in particles:
        if i.get_width() > 0 and i.get_height() > 0:
            self.img.put("#ff0000", (int(i.get_width() *
self.meter_to_pixel), int(i.get_height() *
self.meter_to_pixel)))
        self.old_red_pixels.append((int(i.get_height()),
int(i.get_width())))

    def draw_particles(self, particles):
    self.restore_old_image()
    for particle in particles:
        self.draw_dot(particle.get_width(),
particle.get_height())

self.old_red_pixels.append((int(particle.get_width()),
int(particle.get_height())))
    self.window.update()

    def draw_dot(self, x, y):
    if x > 0 and y > 0:
        self.img.put("#76ff7a", (int(x), int(y)))


    def draw_dot_blue(self, x, y):
    if x > 0 and y > 0:
        self.img.put("#6161ff", (int(x), int(y)))

    def draw_dot_red(self, x, y):
    if x > 0 and y > 0:
        self.img.put("#ff0000", (int(x), int(y)))
```